

SHAALE-VIKAS

Android App Development — Standard Operating Procedure

Complete Development & Implementation Guide

School-Alumni Bridge Platform using GenAI

Project Title	Shaale-Vikas
Project Type	Android Native Application (Kotlin)
Technology Stack	Firebase + Gemini API + Material Design 3
Document Type	Standard Operating Procedure (SOP)
Prepared for	Development Team / Antigravity
Date	May 06, 2026

TABLE OF CONTENTS

1. Executive Summary
2. Project Architecture Overview
3. Technology Stack & Tools
4. Data Model & Database Schema
5. Screen-by-Screen Design Specifications
6. User Authentication & Role Management
7. Feature Implementation Details
8. API Integration & Backend Logic
9. Screen Navigation Flow & Wireframes
10. GenAI Integration Specifications
11. Testing & Quality Assurance
12. Deployment & Release Strategy

1. EXECUTIVE SUMMARY

Shaale-Vikas is a mobile application designed to bridge the gap between rural school administrators and alumni networks. The app enables schools to transparently list infrastructure repair needs, allows alumni to pledge support, and provides real-time progress tracking through Firebase. The application incorporates Generative AI to assist with cost estimation and need analysis.

Key Objectives:

- Enable headmasters to post and manage school repair needs
- Allow alumni to view needs and pledge contributions
- Provide real-time progress tracking and updates
- Integrate GenAI for intelligent cost estimation
- Build community through transparency and impact documentation
- Ensure role-based access control and security

2. PROJECT ARCHITECTURE OVERVIEW

The Shaale-Vikas application follows a three-tier architecture: Frontend (Android UI), Backend (Firebase), and AI Services (Gemini API).

Tier	Component	Technology	Responsibility
Frontend	Android UI/UX	Kotlin + Material Design 3	User interface, navigation, forms, media handling
Backend	Database	Firebase Firestore	Data persistence, real-time sync
Backend	Authentication	Firebase Auth	User login, role management, session
Backend	File Storage	Firebase Cloud Storage	Image uploads (before/after photos)
AI Services	Cost Estimation	Google Gemini API	AI-powered cost suggestions
Networking	API Calls	Retrofit + OkHttp	REST communication with Gemini

3. TECHNOLOGY STACK & TOOLS

- **Android Studio:** Primary IDE for development. Use Kotlin for all code. Min API Level: 24 (Android 7.0). Target API Level: 34+
- **Kotlin:** Modern, concise language for Android. Use Coroutines for async operations. Type-safe, null-safe.
- **Firebase Realtime Database:** NoSQL cloud database. Real-time sync for needs, pledges, progress. Offline persistence enabled.

- **Firebase Firestore:** Alternative/complement to Realtime DB. Better for complex queries. Consider hybrid approach.
- **Firebase Authentication:** Handles login/signup with email/phone/Google Sign-In. Manage user roles (Admin/Alumni).
- **Firebase Cloud Storage:** Store image files (before/after photos). Implement secure download URLs.
- **Google Gemini API:** Generative AI for cost estimation and photo analysis. Called via REST + Retrofit.
- **Retrofit 2 + OkHttp:** HTTP client for REST API calls to Gemini. Handle async requests with Coroutines.
- **Material Design 3:** Google's UI framework. Use predefined components: Cards, FABs, Progress Bars, Navigation Drawer.
- **ImagePicker Library:** Third-party lib (e.g., ucrop or Android native). Access camera/gallery for photo selection.
- **Glide or Coil:** Image loading/caching library. Efficient image display from Firebase Storage URLs.
- **Figma:** UI/UX design tool. Create wireframes and high-fidelity mockups before implementation.
- **Git + GitHub:** Version control. Commit frequently, use feature branches, code reviews.

4. DATA MODEL & DATABASE SCHEMA

This section defines the core data structures stored in Firebase Firestore. All timestamps use server-side timestamps for consistency.

4.1 Users Collection

Path: /users/{userId}

Purpose: Store user profile data and role information

Fields:

Fields:

- **userId** (string): Firebase Auth UID (auto-generated) — Unique identifier
- **phoneNumber** (string): User's phone number — Used for OTP login
- **email** (string): User email — Optional, for email login
- **displayName** (string): User's full name — For hall of fame and profiles
- **role** (string): enum: 'admin' or 'alumni' — Determines app UI/permissions
- **schoolName** (string): Name of school (admin only) — Which school they represent
- **schoolId** (string): Reference to school doc (admin) — Link to school data
- **profilePhoto** (string): URL to profile image — Stored in Cloud Storage
- **createdAt** (timestamp): Account creation timestamp — Server-side timestamp
- **lastLogin** (timestamp): Last login time — For activity tracking

4.2 Needs Collection

Path: /schools/{schoolId}/needs/{needId}

Purpose: Store school repair/improvement needs

Fields:

- **needId** (string): Auto-generated document ID — Unique per need
- **schoolId** (string): Reference to parent school — Which school needs this
- **title** (string): Short title (e.g., 'Roof Repair') — Max 100 chars
- **description** (string): Detailed description — Max 500 chars
- **category** (string): enum: 'roof', 'furniture', 'paint', etc. — For filtering
- **estimatedCost** (number): Cost estimate in INR — Generated by GenAI or admin
- **costDescription** (string): Why this cost? (e.g., '5 sheets of corrugated metal') — Details for transparency
- **beforePhotoUrl** (string): URL to before photo — Firebase Storage URL
- **afterPhotoUrl** (string): URL to after photo (empty until complete) — Added after work completion

- **currentFundsCollected** (number): Sum of all pledges — Auto-calculated from pledges subcollection
- **totalPledges** (number): Count of pledges — For progress calculation
- **progressPercentage** (number): $\text{currentFundsCollected} / \text{estimatedCost} * 100$ — Real-time calculated
- **status** (string): enum: 'open', 'in-progress', 'completed' — Workflow state
- **createdAt** (timestamp): When need was posted — Server-side
- **updatedAt** (timestamp): Last modification time — Server-side

4.3 Pledges Subcollection

Path: /schools/{schoolId}/needs/{needId}/pledges/{pledgeId}

Purpose: Store individual pledge records (simulated donations)

Fields:

- **pledgeId** (string): Auto-generated ID — Unique per pledge
- **userId** (string): Who pledged — Reference to alumni user
- **amount** (number): Pledge amount (simulated) — For display on hall of fame
- **pledgeType** (string): enum: 'item' or 'fund' — Item donation or money
- **itemDescription** (string): What item being pledged (if applicable) — E.g., '5 gallons paint'
- **status** (string): enum: 'pending', 'confirmed', 'delivered' — Pledge workflow
- **createdAt** (timestamp): When pledge was made — Server-side
- **completedAt** (timestamp): When pledge was fulfilled — When item/fund delivered

4.4 Reviews & Ratings Collection

Path: /schools/{schoolId}/reviews/{reviewId}

Purpose: Store alumni reviews of completed work

Fields:

- **reviewId** (string): Auto-generated ID — Unique per review
- **needId** (string): Which need is being reviewed — Reference to completed need
- **userId** (string): Who wrote the review — Reference to alumni user
- **rating** (number): 1-5 star rating — Integer from 1 to 5
- **comment** (string): Review text — Max 300 chars
- **createdAt** (timestamp): When review was posted — Server-side

5. SCREEN-BY-SCREEN DESIGN SPECIFICATIONS

Detailed UI/UX specifications for each screen. Includes layout, components, colors, interactions, and backend connections.

SCREEN 1: SPLASH / WELCOME SCREEN

Purpose: First screen when app launches. Shows app branding and asks user role.

Duration: 2-3 seconds of splash animation, then transition.

Layout: Full-screen with logo centered.

Components:

- Shaale-Vikas Logo (centered, 200x200dp)
- App tagline: 'School-Alumni Bridge Platform'
- Two prominent buttons: 'I'm a Headmaster' | 'I'm an Alumni'
- Bottom text: 'Powered by GenAI'

Interactions: User taps one of the two buttons → Navigate to login screen with pre-selected role.

Backend: No backend calls yet. Navigation only.

SCREEN 2: LOGIN / PHONE AUTHENTICATION

Purpose: Authenticate user with phone number and OTP. Route to dashboard based on role.

Components:

- Back button (to go back to role selection)
- Title: 'Login with Phone Number'
- Role indicator: 'Logging in as: [Headmaster/Alumni]'
- Phone input field (country code + number)
- Send OTP button
- OTP input field (6 digits, appears after OTP sent)
- Verify OTP button
- Forgot password / Signup link

Flow:

1. User enters phone number
2. Click 'Send OTP' → Firebase Auth sends OTP via SMS
3. OTP field appears
4. User enters 6-digit OTP
5. Click 'Verify OTP' → Firebase verifies → Create/fetch user record from Firestore
6. Navigate to appropriate dashboard (Admin or Alumni)

Firestore Integration: Firebase Authentication (Phone OTP), then query `/users/{userId}` from Firestore to get role and redirect.

SCREEN 3: ADMIN DASHBOARD (HEADMASTER)

Purpose: Headmaster's main hub. View all school needs, create new needs, monitor progress, upload impact photos.

Components:

- Top AppBar with school name + user profile icon
- Tab 1: 'My Needs' (list of all posted needs)
- Tab 2: 'Analytics' (total pledges, % completion, donor count)
- Tab 3: 'Settings' (profile, school info, logout)
- Floating Action Button (FAB): 'New Need' (blue, bottom-right)
- For each need in list: Card showing title, cost, progress bar (%), status badge

Card Details (each need):

- Need title (bold, large text)
- 'Cost: █X,XXX' and 'Collected: █Y,YYY (Z%)'
- Progress bar (Material Design)
- Status chip: Open / In-Progress / Completed
- Click card → Open Need Detail screen

Backend: Query `/schools/{schoolId}/needs` from Firestore. Real-time listener for live updates.

SCREEN 4: CREATE / EDIT NEED (ADMIN)

Purpose: Form for headmaster to create a new repair need or edit existing one.

Form Fields:

- Need Title (text input, max 100 chars, required)
- Description (multiline text, max 500 chars, required)
- Category (dropdown: 'Roof', 'Furniture', 'Paint', 'Electrical', 'Plumbing', 'Other')
- Estimated Cost (number input, currency █, required)
- Cost Details (text, e.g., 'Materials: 5 sheets corrugated metal @ █500 each')
- 'Upload Before Photo' button (opens camera/gallery via ImagePicker)
- Before photo preview (thumbnail)
- 'Get AI Cost Suggestion' button → Calls Gemini API
- AI-generated cost appears in Estimated Cost field
- 'Save Need' button
- 'Cancel' button

Interactions:

- Click 'Upload Before Photo' → ImagePicker → Compress image → Upload to Firebase Storage → Store URL in form
- Click 'Get AI Cost Suggestion' → Call Gemini API with need description → Display suggestion in cost field
- Click 'Save Need' → Validate form → Create Firestore document → Toast: 'Need posted successfully'

Backend: POST to `/schools/{schoolId}/needs` with form data. Upload image to Cloud Storage. Call Gemini API for cost estimation.

SCREEN 5: NEED DETAIL SCREEN (ADMIN VIEW)

Purpose: Detailed view of a single need with all pledges, progress, and ability to upload after-photo.

Components:

- Back button + Title of need (top)
- Before photo (large, clickable to expand)
- Need title, description, category, status
- Cost section: 'Estimated: █X,XXX' | 'Collected: █Y,YYY' | 'Progress: Z%'
- Progress bar (animated, updates in real-time)
- Cost details explanation (read-only text)
- Pledges section: List of all pledges (expandable list)
- Each pledge: Donor name, amount/item, status (pending/confirmed/delivered)
- 'Upload After Photo' button (appears when status = in-progress)
- After photo preview (if uploaded)
- Edit button (pencil icon, top-right) → Goes to Edit screen
- Mark as Complete button (status → 'Completed')

Real-time Updates: Listen to Firestore changes for pledges & progress. Update UI instantly.

Backend: Real-time listener on `/schools/{schoolId}/needs/{needId}` and `/schools/{schoolId}/needs/{needId}/pledges`.

SCREEN 6: ALUMNI DASHBOARD (NEEDS DIRECTORY)

Purpose: Alumni home screen. Browse all school needs, filter by category, search for specific needs.

Components:

- Top AppBar: 'Shaale-Vikas' + user profile icon
- Search bar (search needs by title or school)
- Filter chips: 'All', 'Roof', 'Furniture', 'Paint', 'Electrical', 'Other'
- Scrollable list of need cards (infinite scroll / pagination)

- Each card shows: School name, need title, cost, progress bar, status
- Tap card → Open Need Detail (Alumni view)
- Bottom navigation: Home | Pledges (my pledges) | Hall of Fame | Profile

Card Layout (each need):

- School name (small, muted text)
- Before photo (thumbnail, left side)
- Need title (bold)
- Cost: ■X,XXX | Collected: ■Y,YYY (Z%)
- Progress bar
- Status badge

Backend: Query all schools and their needs from Firestore. Implement pagination for performance.

SCREEN 7: NEED DETAIL SCREEN (ALUMNI VIEW)

Purpose: Detailed view of a need for alumni. Shows full details, photo, and ability to pledge or message vendor.

Components:

- Back button + School name (top)
- Before photo (full-width, clickable to expand/zoom)
- Need title, description, category, status
- School info card: School name, location, headmaster name, headmaster phone
- Cost breakdown: 'Estimated: ■X,XXX' | 'Collected: ■Y,YYY' | 'Progress: Z%'
- Progress bar (animated)
- Cost details (why this cost?)
- Pledges summary: 'X pledges from Y donors'
- Hall of Fame mini: Top 5 donors (names, amounts) with avatars
- 'Pledge Now' button (primary, blue)
- 'Message via WhatsApp' button (green, WhatsApp icon)
- 'Read Reviews' section (star rating + snippets, if available)

Pledge Button Interaction:

1. Click 'Pledge Now'
2. Modal/Dialog appears: 'What would you like to pledge?'
3. Radio options: 'Money' | 'Item'
4. If Money: Enter amount (number input)
5. If Item: Enter item description (text)
6. 'Confirm Pledge' button → Create pledge document in Firestore → Show toast: 'Pledge recorded!'

WhatsApp Button Interaction:

- Click → Open WhatsApp with pre-filled message (Intent): 'Hi, I'm interested in supporting this need [need title]'

at [school name]. Can we discuss?'

- Message goes to headmaster's phone number (from school data)

Backend: Real-time listener on need details, pledges subcollection. Fetch school info and headmaster phone.

SCREEN 8: HALL OF FAME

Purpose: Public leaderboard of all donors sorted by total contribution or recent pledges.

Components:

- Sorting options: 'Total Contribution' (default) | 'Recent Pledges'
- Scrollable list of donors with rank number, profile photo, name, total amount pledged, badge (if top 3)
- Current user's rank highlighted
- Tap donor → View their profile & pledging history

Backend: Aggregate pledges by userId across all schools/needs. Real-time calculation.

SCREEN 9: MY PLEDGES (ALUMNI)

Purpose: Alumni view all their pledges across all schools, track status.

Components:

- Tabs: 'Pending' | 'Confirmed' | 'Delivered'
- List of pledges with: School name, need title, amount/item, status, date pledged
- Tap pledge → See need detail + pledge details
- Filters by status (default: All)

Backend: Query pledges where userId = current user from across all schools/needs.

SCREEN 10: REVIEWS & RATINGS

Purpose: Alumni can rate and review completed needs. Reviews appear on need detail screen.

Components:

- List of completed needs (only completed needs can be reviewed)
- Each card: School, need title, 'Leave Review' button
- Click 'Leave Review' → Modal/Screen with form:
 - - Star rating (1-5, interactive)
 - - Comment text (multiline, max 300 chars)
 - - 'Submit Review' button
- After submit → Toast: 'Thank you for your feedback!'
- Review appears on need detail screen (visible to all users)

Backend: POST review to /schools/{schoolId}/reviews. Aggregate rating for need display.

SCREEN 11: USER PROFILE

Purpose: View and edit user profile. Different content for admin vs. alumni.

Admin Profile Components:

- Profile photo (editable, tap to change)
- Name (editable text field)
- School name (editable text field)
- School contact info (editable)
- Role badge: 'Headmaster'
- Statistics: Total needs posted, total funds collected, total donors
- Edit button (pencil icon)
- 'Save Changes' button (appears after edit)
- 'Change Password' button
- 'Logout' button (bottom, destructive red)

Alumni Profile Components:

- Profile photo (editable)
- Name (editable)
- Email/Phone (display only)
- Role badge: 'Alumni Donor'
- Statistics: Total pledges, total amount pledged, schools supported
- Recent pledges (mini list, last 5)
- Edit button
- 'Save Changes' button
- 'Change Password' button
- 'Logout' button

Backend: UPDATE /users/{userId} on Firestore. Profile photo uploaded to Cloud Storage.

6. USER AUTHENTICATION & ROLE MANAGEMENT

Authentication Flow:

1. User selects role (Admin/Alumni)
2. Firebase Phone Auth: Send OTP via SMS
3. User enters OTP → Firebase verifies
4. Firebase creates UID automatically
5. App creates/updates user doc in Firestore with role
6. Navigate to appropriate dashboard based on role
7. Session persists (Firebase handles token refresh)

Role-Based Access Control (RBAC):

Feature	Admin (Headmaster)	Alumni (Donor)
View Needs	Own school needs only	All schools & needs
Create Need	Yes	No
Edit Need	Own needs only	No
Upload Photos	Before & After	No
View Pledges	All pledges on own needs	Own pledges only
Pledge on Need	No	Yes
Rate/Review	No (conflict)	Yes
Hall of Fame	View only	View & appear
Admin Dashboard	Yes	No

Firestore Security Rules:

Rules enforce role-based access at database level:

- Admin can only read/write own school's needs
- Alumni can only write pledges (not edit them)
- Alumni can read all needs from all schools
- Only authenticated users can access data

7. FEATURE IMPLEMENTATION DETAILS

7.1 Real-Time Synchronization

Requirement: Progress bars, pledge lists, and hall of fame update instantly when data changes.

Implementation: Use Firestore real-time listeners (onSnapshot) for collections and documents. Wrap in ViewModel and LiveData/StateFlow for lifecycle-safe updates.

Code Pattern:

```
db.collection('schools').document(schoolId).collection('needs')
.addSnapshotListener { snapshot, error ->
```

```
// Update UI when data changes  
}
```

7.2 Photo Upload & Management

Flow:

1. User clicks 'Upload Photo'
2. ImagePicker opens (camera or gallery)
3. User selects/captures image
4. Compress image (max 2MB)
5. Generate unique filename: {needId}_{timestamp}.jpg
6. Upload to Firebase Storage: /schools/{schoolId}/needs/{needId}/before.jpg
7. Get download URL → Store in Firestore need doc
8. Display in UI with Glide/Coil

7.3 Progress Calculation

Formula: $\text{Progress\%} = (\text{Sum of pledge amounts} / \text{Estimated Cost}) \times 100$

Implementation: Cloud Function (optional) or calculated in app from pledges subcollection

Update Timing: Recalculate whenever a new pledge is created

Display: ProgressBar component with percentage text, animated transitions

7.4 WhatsApp Integration

Implementation: Use Android Intent to open WhatsApp with pre-filled message

Code:

```
val message = "Hi, I'm interested in supporting [need title]. Can we discuss?" val intent =  
Intent(Intent.ACTION_VIEW).apply { data =  
Uri.parse("https://wa.me/91${phoneNumber}?text=${URLEncoder.encode(message)}") }  
startActivity(intent)
```

8. API INTEGRATION & BACKEND LOGIC

8.1 Gemini API for Cost Estimation

Endpoint: <https://generativelanguage.googleapis.com/v1beta/models/gemini-pro:generateContent>

Method: POST

Request Body:

```
{ "contents": [{ "parts": [{ "text": "Estimate the cost to repair: [NEED_DESCRIPTION].  
Respond with only a number in INR." }] }] }
```

Response: JSON with generated text containing estimated cost

Android Implementation: Use Retrofit to make HTTP POST call. Parse response JSON. Display in UI.

Error Handling: If API fails, show toast: 'Could not generate estimate. Enter manually.' Catch network errors.

8.2 Firebase Realtime Database Queries

- **Get all needs for a school:** Query /schools/{schoolId}/needs
- **Get all pledges for a need:** Real-time listener on /schools/{schoolId}/needs/{needId}/pledges
- **Get all pledges by a user:** Query /pledges?userId=X across all schools
- **Get top donors (Hall of Fame):** Aggregate pledges by userId, sort by total amount
- **Get all completed needs:** Query /needs?status='completed'
- **Calculate progress for a need:** Sum amounts in pledges subcollection / estimatedCost

8.3 Firebase Storage for Images

- **Upload image:** PUT request to /schools/{schoolId}/needs/{needId}/before.jpg
- **Get download URL:** Firebase generates persistent HTTPS URL
- **Cache images:** Glide/Coil handles caching automatically
- **Delete image:** Called when need is deleted or image is replaced

9. SCREEN NAVIGATION FLOW & WIREFRAMES

This section describes how screens connect and user navigation paths.

Navigation Hierarchy:

SPLASH SCREEN ■■ "I'm a Headmaster" ■■→ LOGIN (role=admin) ■ ■■ Auth Success ■■→ ADMIN DASHBOARD ■
■ ■■ Tap FAB ■■→ CREATE NEED ■ ■ ■■ Tap Card ■■→ NEED DETAIL (Admin) ■ ■ ■■ Tap Edit ■■→ EDIT
NEED ■ ■ ■■ Tap Upload After ■■→ PHOTO UPLOAD ■ ■ ■■ Tap Analytics ■■→ ANALYTICS SCREEN ■ ■ ■■
Tap Settings ■■→ PROFILE ■ ■■ Auth Fail ■■→ ERROR MESSAGE ■ ■■ "I'm an Alumni" ■■→ LOGIN
(role=alumni) ■■ Auth Success ■■→ ALUMNI DASHBOARD ■ ■■ Tap Search ■■→ SEARCH RESULTS ■ ■■ Tap
Filter ■■→ FILTERED LIST ■ ■■ Tap Need Card ■■→ NEED DETAIL (Alumni) ■ ■ ■■ Tap Pledge ■■→ PLEDGE

DIALOG ■ ■ ■ Tap WhatsApp ■■→ EXTERNAL (WhatsApp) ■ ■■ Bottom Nav "Pledges" ■■→ MY PLEDGES ■ ■■
Bottom Nav "Hall of Fame" ■■→ LEADERBOARD ■ ■■ Bottom Nav "Reviews" ■■→ REVIEWS SCREEN ■ ■■ Bottom
Nav "Profile" ■■→ PROFILE ■■ Auth Fail ■■→ ERROR MESSAGE

Tab Navigation (within dashboards):

Admin Dashboard: Top tab bar with 3 tabs: 'My Needs', 'Analytics', 'Settings'

Alumni Dashboard: Bottom navigation bar with 5 items: 'Home', 'Pledges', 'Hall of Fame', 'Reviews', 'Profile'

Need Detail (Alumni): ScrollableLayout with all sections stacked vertically

10. GENAI INTEGRATION SPECIFICATIONS

10.1 Use Cases for Gemini API

- **Cost Estimation:** When admin enters a need description (e.g., 'Repair leaking roof'), call Gemini to estimate cost. User can accept or override.
- **Damage Assessment:** If admin uploads a photo of damage, call Gemini with Vision API to analyze damage severity and suggest repair approach.
- **Need Description Enhancement:** Admin enters rough description → Gemini refines it into more detailed, persuasive text for alumni to read.
- **Impact Summarization:** After work is completed, upload before/after photos → Gemini generates a summary of impact ('Successfully repaired X, benefiting Y students').

10.2 Gemini API Setup & Authentication

Step 1: Get API key from Google Cloud Console (free tier available)

Step 2: Store API key in Android BuildConfig (or secure key store for production)

Step 3: Add Retrofit dependency for HTTP calls

Step 4: Create Retrofit service interface for Gemini endpoints

Step 5: In onCreate or module, initialize Retrofit with base URL:
<https://generativelanguage.googleapis.com/v1beta/>

10.3 Example: Cost Estimation Code

```
// In ViewModel or Repository suspend fun generateCostEstimate(description: String): String
{ val prompt = "Estimate cost in INR to: $description. Respond with number only." val
  request = GeminiRequest( contents = listOf(Content( parts = listOf(Part(text = prompt)) ))
) return geminiService.generateContent(API_KEY, request) .contents[0].parts[0].text } // In
UI viewModel.generateCostEstimate(needDescription).collect { estimate ->
costInput.setText(estimate) showToast("AI suggestion: ■$estimate") }
```

10.4 Error Handling for GenAI

- Network timeout: Show message 'Estimate unavailable. Enter manually.'
- Invalid API key: Log error, show message

- Rate limit exceeded: Queue request, retry with exponential backoff
- Unexpected response format: Parse safely with try-catch, default to user input

11. TESTING & QUALITY ASSURANCE

11.1 Unit Testing

- ViewModel logic (progress calculation, sorting donors)
- Data models (serialization, validation)
- Utility functions (image compression, date formatting)
- Progress percentage calculation
- Role-based access checks

11.2 Integration Testing

- Firebase Auth login/logout flow
- Firestore read/write (create need, pledge, review)
- Firebase Storage image upload/download
- Gemini API calls (mock + real)
- Navigation between screens

11.3 UI/UX Testing

- Real-time updates (pledge triggers UI refresh instantly)
- Offline behavior (if offline, show cached data)
- Image loading (avoid janky layout shifts)
- Progress bar animation (smooth, not jarring)
- Responsive design on multiple screen sizes (phone + tablet)
- Accessibility (text sizes, contrast, dark mode support)

11.4 Manual Testing Checklist

- [] Admin can create, edit, delete needs
- [] Alumni can view all needs and filter by category
- [] Pledge system records and updates progress correctly
- [] Hall of Fame ranks donors accurately
- [] Reviews appear on need detail screen
- [] WhatsApp integration launches with correct phone number
- [] Photo uploads work on slow networks

- ☐ Real-time updates work across multiple devices
- ☐ Logout clears session and returns to login
- ☐ GenAI cost estimation works and provides sensible results

12. DEPLOYMENT & RELEASE STRATEGY

12.1 Pre-Release Checklist

- ☐ All features implemented per specification
- ☐ Unit tests pass (>80% code coverage target)
- ☐ Integration tests pass on real Firebase project
- ☐ Manual testing completed (see checklist above)
- ☐ Performance testing (app launches in <3 seconds)
- ☐ Security review (API keys in BuildConfig, Firestore rules validated)
- ☐ Crash reporting setup (Crashlytics)
- ☐ Analytics tracking added (Firebase Analytics)
- ☐ Privacy policy written and included in app
- ☐ Target API level ≥ 33 (per Google Play requirements 2024)

12.2 Google Play Store Submission

1. Generate signed APK/AAB (Android App Bundle)
2. Create Google Play Developer account (one-time fee ■799)
3. Upload AAB to Play Console
4. Fill app description, screenshots, privacy policy
5. Set content rating (rate for age groups)
6. Request review (Google reviews in 24-48 hours)
7. Once approved, app is live on Play Store

12.3 Production Firebase Setup

Database Backup: Enable automatic daily backups in Firestore

Security Rules: Activate Firestore security rules (enforce authentication, role-based access)

Monitoring: Set up Firebase monitoring for read/write latency

Costs: Monitor Firebase usage (Realtime DB, Cloud Storage, Gemini API calls)

12.4 Post-Release & Updates

Phase 1 (Week 1-2): Monitor crash reports, fix critical bugs, engage with users for feedback

Phase 2 (Month 2-3): Add minor features based on user feedback, optimize performance

Phase 3 (Month 4+): Plan next major version with expanded features

CONCLUSION & NEXT STEPS

This SOP document provides comprehensive specifications for the Shaale-Vikas Android application. Development teams should follow these specifications closely to ensure alignment with project requirements and vision. Regular communication between stakeholders and the development team is essential for addressing ambiguities and adapting to changing requirements.

Immediate Next Steps:

1. Set up Firebase project and enable required services (Auth, Firestore, Storage, Realtime DB)
2. Create Android project in Android Studio with Kotlin
3. Add Firebase SDK and Retrofit dependencies
4. Design wireframes in Figma (visual reference)
5. Implement authentication screens and login flow
6. Build database schema in Firestore (test with sample data)
7. Implement admin dashboard screens
8. Implement alumni dashboard screens
9. Integrate Gemini API for cost estimation
10. Implement real-time listeners and progress calculations
11. Add image upload functionality
12. Integrate WhatsApp messaging
13. Build reviews & ratings system
14. Comprehensive testing and bug fixes
15. Prepare for Google Play Store submission

This document was generated on May 06, 2026 at 05:34. It serves as the definitive technical specification for the Shaale-Vikas project.